SafeHire: An AI-Powered Service Hiring Platform

CSE299: JUNIOR DESIGN COURSE
Section- 4 (Group 7)
Nazia Faruque Diya
ID: 2222666042
Afridur Rahman Khan Mim
ID: 2231521642
Department of Electrical and Computer Engineering
North South University, Bangladesh

***Abstract*—SafeHire is a web-based AI-assisted service hiring platform designed to improve the efficiency, security, and transparency of hiring service workers. Traditional hiring systems suffer from major issues such as lack of worker verification, inefficient matching, and absence of structured workflows. SafeHire addresses these problems by integrating a structured worker profile system, admin-based verification, AI-assisted job-worker matching, complete hiring workflow, and transaction tracking. The platform uses a rule-based intelligent matching system that evaluates skills, profession, location, experience, and ratings to recommend optimal candidates. Built using Flask, SQLite, and Tailwind CSS, the system demonstrates strong modular design, scalable architecture, and real-world applicability.**

***Index Terms*—SafeHire, AI-assisted matching, worker verification, service hiring platform, Flask, SQLite, Tailwind CSS, recommendation system**

I. Introduction

The demand for service workers such as electricians, cleaners, plumbers, and technicians is increasing in modern society. However, hiring these workers remains inefficient and risky due to:

- Lack of identity verification
- No structured hiring process
- Difficulty in finding reliable workers
- Absence of intelligent recommendation systems

Users often rely on informal sources such as social media, personal contacts, or unverified recommendations. This may lead to unreliable outcomes and safety concerns.

SafeHire introduces a centralized digital solution that:

- Ensures worker verification
- Uses AI-assisted matching
- Provides structured hiring workflow
- Improves transparency and trust
- Supports worker, employer, and admin roles

II. Problem Statement

The major problems identified are:

1) **Unverified Workers**
   - No identity validation
   - Increased safety risk
   - Lack of trust between employer and worker
2) **Inefficient Searching**
   - Users manually search through contacts or social media
   - Searching is time-consuming
   - Results are often unreliable
3) **No Intelligent Matching**
   - No system suggests the best worker automatically
   - Worker skill, profession, experience, and rating are not properly considered
4) **Lack of Workflow**
   - No proper system for job posting
   - No clear flow for apply, hire, payment, and review
5) **No Transparency**
   - Users cannot easily track hiring status
   - Worker performance is not always visible
   - Transaction status is often unclear

These limitations justify the need for SafeHire.

III. Proposed Solution

SafeHire solves these issues through a centralized web-based service hiring platform.

A. Key Features

- Worker verification system
- AI-assisted matching system
- Job posting and application system
- Hiring workflow system
- Transaction management
- Payment simulation
- Review and rating system
- Admin monitoring dashboard

The system integrates all stages of hiring into a single platform.

IV. Use Case Diagram of SafeHire

The use case diagram represents the main actors and their system activities.

A. Actors

- Worker
- Employer
- Admin

B. Worker Use Cases

- Sign up / login
- Create worker profile
- Update profile
- View AI-matched jobs
- Apply for jobs
- View applications
- View reviews

C. Employer Use Cases

- Sign up / login
- Post job
- View AI worker matches
- Hire verified worker
- Complete payment
- Submit review

D. Admin Use Cases

- Admin login
- Verify or reject workers
- View employers
- View jobs
- Monitor transactions
- Monitor reviews

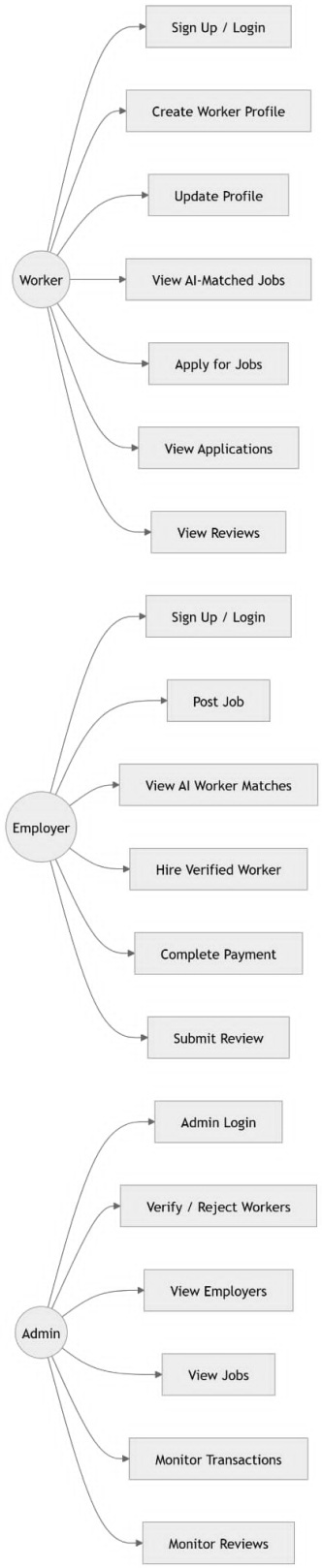

Fig. 1: Use Case Diagram of SafeHire

V. System Architecture

The system follows a client-server architecture.

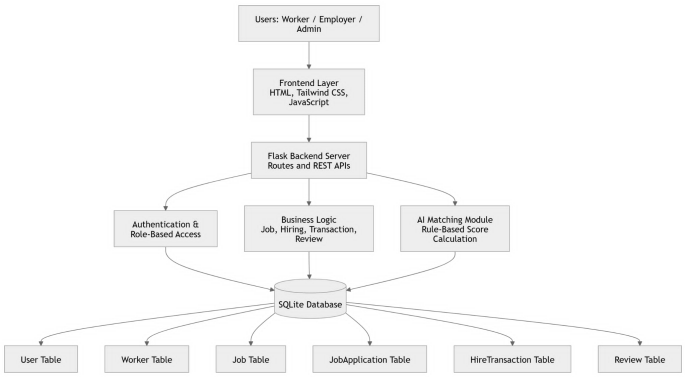

Fig. 2: System Architecture of SafeHire

A. Main Components

1) **Frontend**
   - HTML
   - Tailwind CSS
   - JavaScript
   - Handles user interface
2) **Backend**
   - Flask
   - Handles routes and REST APIs
   - Processes business logic
   - Manages role-based access
3) **Database**
   - SQLite
   - Stores users, workers, jobs, transactions, and reviews
4) **AI Matching Module**
   - Performs rule-based score calculation
   - Matches jobs with suitable workers
   - Sorts recommendations by score

VI. System Workflow

The workflow of SafeHire includes the following steps:

1) Worker registers
2) Worker creates or updates profile
3) Worker submits NID, skills, and profession
4) Admin verifies worker
5) Employer posts job
6) AI matching module suggests workers
7) Employer views applicants
8) Employer hires worker
9) Job status becomes assigned
10) Transaction is created
11) Employer completes payment
12) Job status becomes completed
13) Employer submits review
14) Worker rating is updated

This ensures a complete hiring lifecycle.

Fig. 3: Overall System Flow Diagram of SafeHire

VII. Module-Wise Implementation

A. Worker Management System

Workers create profiles with:

- Name
- NID
- Phone number
- Address
- Profession
- Skills
- Experience

1) Create Profile Page

- Structured form ensures data consistency
- Required fields prevent missing data
- Worker profile is stored in the database
- Initial worker status is set to pending

2) Admin Verification

Admin verifies workers using three possible status values:

- Pending
- Verified
- Rejected

If approved:

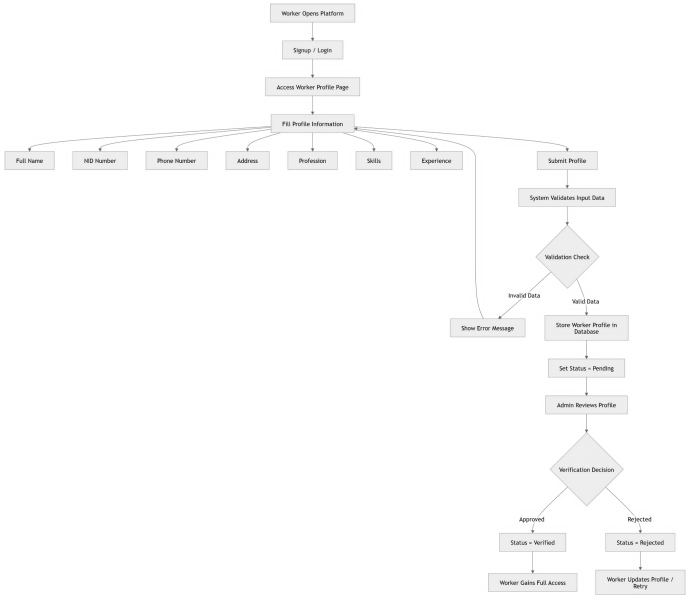

Fig. 4: Worker Registration and Verification Flow

- Worker status becomes verified
- Worker gains full access

If rejected:

- Worker status becomes rejected
- Worker can update profile and retry

B. AI-Based Matching System

The system uses rule-based scoring.

TABLE I: AI Matching Factors and Weight Distribution in SafeHire

Matching Factor	Description	Score Contribution
Profession Match	Checks if worker profession matches job category exactly.	+35 points
Skill Match	Compares worker skills with job title, category, and description.	+15 points
Keyword Matching	Matches important keywords between job title and worker skill set.	+10 points
Location Proximity	Checks if worker and job location are similar or nearby.	+5 to +15 points
Experience Level	Evaluates years of experience of the worker.	+5 to +20 points
Worker Rating	Uses average rating to prioritize high-quality workers.	+5 to +25 points
Normalization Factor	Converts text into comparable format.	Applied before scoring
Final Score Ranking	Combines all scores to rank workers.	0–100+ score range

1) Matching Process

- Extract job features
- Retrieve worker data
- Normalize text data
- Filter workers

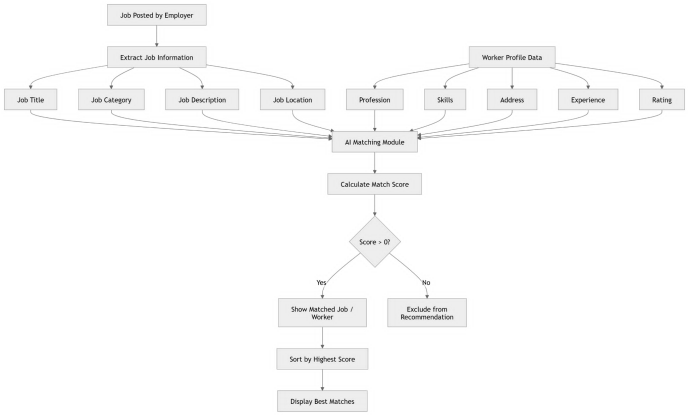

Fig. 5: AI Matching Flow Diagram

- Calculate match score
- Rank workers by score
- Show top matches

2) Benefits

This improves:

- Speed
- Accuracy
- User experience
- Transparency
- Explainability

C. Job Posting and Application System

Employers can:

- Create jobs
- Add job title
- Select job category
- Enter job location
- Add budget
- Write job description

Workers can:

- View available jobs
- View AI-matched jobs
- Apply for jobs
- Track application status

D. Hiring Workflow System

The hiring workflow follows this process:

- Job is posted
- Job status is set to open
- Employer views applicants or AI matches
- Employer selects worker
- Job status becomes assigned
- Transaction is created
- Payment is completed
- Job status becomes completed
- Employer submits review

The workflow ensures:

- Structured hiring

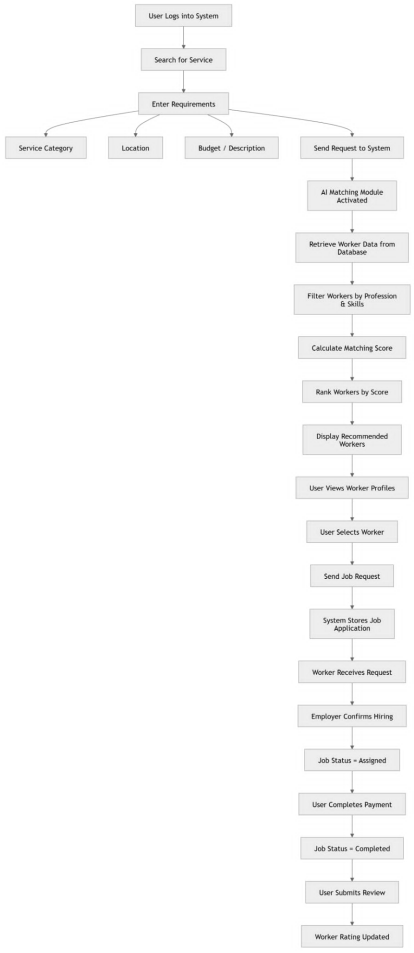

Fig. 6: User Hiring Flow Diagram

- Controlled access
- Verified worker selection
- Proper transaction tracking
- Review-based trust building

E. Transaction Management System

The transaction system tracks:

- Employer name
- Worker name
- Job details
- Payment amount
- Transaction status
- Created date

It includes:

- Status badges
- Assigned status
- Completed status
- Payment simulation
- Review integration

F. Payment Simulation System

The payment system includes:

- "Pay Now" button
- Simulated payment operation
- Transaction status update
- Job status update

After payment:

- Transaction status becomes completed
- Job status becomes completed

G. Review and Rating System

After job completion:

- Employer gives rating from 1 to 5
- Employer adds comments
- Review is stored in database
- Rating is displayed in worker dashboard
- Worker average rating is updated

Benefits:

- Builds trust
- Helps future hiring
- Improves worker credibility
- Supports quality control

H. Admin Dashboard

Admin can:

- View total workers
- View total jobs
- View total transactions
- View reviews
- Manage worker verification
- View employers
- Monitor platform records

The admin dashboard acts as:

- System controller
- Monitoring hub
- Verification authority

I. User Interface Design

From the UI pages, SafeHire includes:

- Dark gradient theme
- Tailwind CSS styling
- Responsive layout
- Clean cards and dashboards
- Role-based navigation
- Status badges

Examples:

- Home page shows stats and workflow
- Worker dashboard shows jobs and reviews
- Employer dashboard shows posted jobs and matches
- Admin dashboard shows monitoring data

Design ensures:

- Modern look
- Easy navigation
- Good user experience
- Responsive interface

J. Backend Development

Technologies:

- Flask
- Flask-SQLAlchemy
- Flask-Login
- Flask-Migrate

Responsibilities:

- API handling
- Authentication
- Session management
- Data validation
- Business logic
- Role-based access control

K. Database Design

Entities:

- User
- Worker
- Job
- JobApplication
- HireTransaction
- Review

Relationships:

- User to Worker
- User to Job
- Worker to JobApplication
- Job to JobApplication
- Job to HireTransaction
- HireTransaction to Review
- Worker to Review
- Employer to Review

Ensures:

- Data consistency
- Efficient queries
- Organized records
- Relational integrity

L. Technologies Used in SafeHire Platform

Category	Technology Used	Purpose / Description
Programming Language	Python	Core backend development
Web Framework	Flask	Handles routing, APIs, and server-side logic
Database	SQLite	Stores users, workers, jobs, transactions, and reviews
ORM	Flask-SQLAlchemy	Database interaction using models
Migration Tool	Flask-Migrate	Handles database schema updates
Authentication	Flask-Login	Manages user sessions and login system
Frontend Structure	HTML5	Builds web page structure
Styling Framework	Tailwind CSS	Provides modern responsive UI design
Scripting Language	JavaScript	Handles dynamic frontend interactions
API Communication	Fetch API	Sends/receives data between frontend and backend
Development Tools	VS Code	Code editor for development
Version Control	Git & GitHub	Code management and collaboration
Deployment (Local)	Flask Development Server	Runs application locally

Fig. 8: Technologies Used in SafeHire Platform

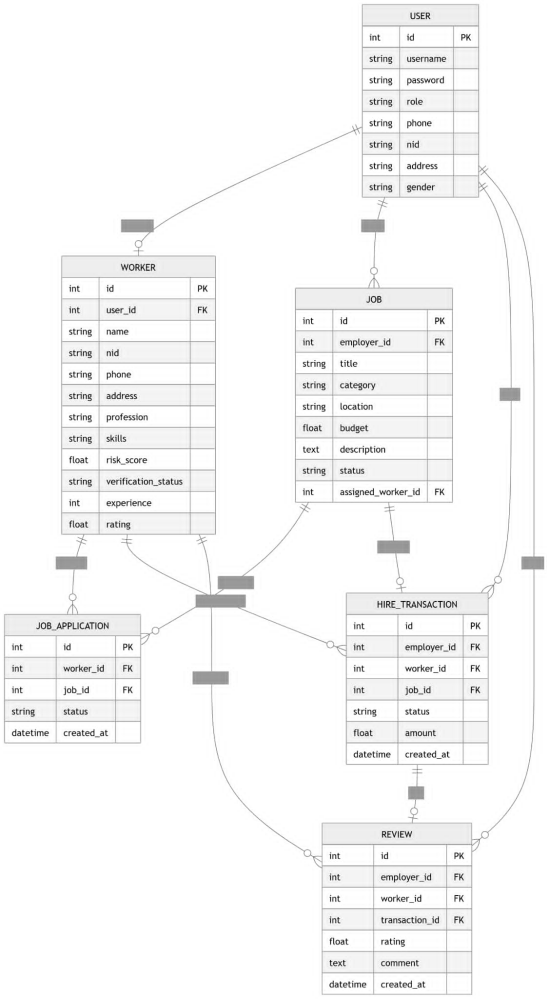

Fig. 7: Entity Relationship Diagram

M. Sequence Diagram of SafeHire

N. System Integration and Testing

Testing confirmed:

- Correct login/signup flow
- Accurate AI matching
- Proper role-based access
- Error handling

O. Comparison of SafeHire with Existing Platforms

SafeHire provides:

- Admin-controlled verification system
- Intelligent rule-based AI matching
- Detailed skill and profession matching
- Fully structured hiring workflow
- High transparency with status tracking
- Full admin monitoring system
- Customizable job posting
- Transparent scoring-based matching
- Academic and research adaptability

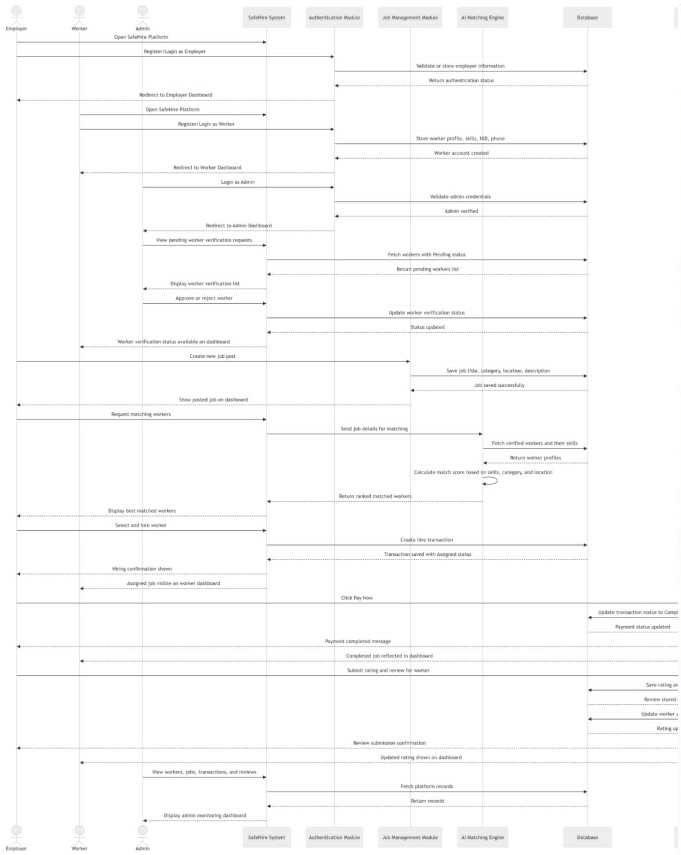

Fig. 9: Sequence Diagram of SafeHire

Features	TaskRabbit	Sheba.xyz	SafeHire (Proposed System)
Worker Verification	⚠ Platform-based verification	⚠ Company-managed verification	✅ Admin-controlled verification system
AI-Based Matching	⚠ Basic filtering (category/location)	❌ Mostly manual assignment	✅ Intelligent rule-based AI matching
Skill-Based Recommendation	⚠ Limited	❌ Not prominent	✅ Detailed skill & profession matching
Hiring Workflow Structure	⚠ Semi-structured	⚠ Service-request based	✅ Fully structured (Job → Apply → Hire → Pay → Review)
Transparency	⚠ Moderate	⚠ Moderate	✅ High transparency (status tracking)
Transaction Tracking	✅ Available	✅ Available	✅ Structured with status (Assigned, Completed)
Payment System	✅ Integrated	✅ Integrated	⚠ Simulated (for academic/demo purpose)
Review & Rating System	✅ Available	✅ Available	✅ Integrated with worker dashboard
Admin Control	❌ Limited visibility	⚠ Centralized control	✅ Full admin monitoring system
User Interface	✅ Professional UI	✅ Mobile-focused UI	✅ Modern responsive UI (Tailwind-based)
Custom Job Posting	⚠ Limited flexibility	❌ Predefined services	✅ Fully customizable job posting
Worker Profile Details	⚠ Basic profile	⚠ Limited details	✅ Detailed profile (skills, experience, rating)
AI Explainability	❌ Not visible	❌ Not available	✅ Transparent scoring-based matching
Academic/Research Adaptability	❌ Commercial system	❌ Commercial system	✅ Designed for extensibility & research

Fig. 10: Comparison of SafeHire with Existing Platforms

VIII. Results and Discussion

The system achieves:

- Faster hiring process
- Verified worker ecosystem
- Transparent workflow
- Structured job posting
- AI-assisted recommendation
- Transaction tracking
- Review-based trust system

Compared to traditional systems, SafeHire is:

- More secure
- More efficient
- More structured
- More transparent
- More suitable for academic demonstration

A. Key Outcomes

- Workers can create verified profiles.
- Employers can post custom jobs.
- AI module can rank suitable workers.
- Admin can monitor platform activity.
- Transactions and reviews can be tracked.

IX. Limitations

The current system has the following limitations:

- AI is rule-based, not machine-learning based
- No real payment gateway
- No real-time chat
- SQLite limits scalability
- Local deployment only
- No mobile app version
- Limited advanced location tracking

X. Future Improvements

Future improvements include:

- Machine learning-based recommendation
- Mobile app version
- Payment gateway integration
- Real-time chat system
- Cloud deployment
- Notification system
- Advanced location-based recommendation
- Improved admin analytics

XI. Conclusion

SafeHire successfully implements a complete service hiring ecosystem with:

- Worker verification
- AI-assisted matching
- Structured hiring workflow

- Transparent transaction system
- Review and rating system
- Admin monitoring

The system demonstrates:

- Strong backend architecture
- Clean UI design
- Database-driven workflow
- Real-world applicability
- Academic extensibility

Although the AI module is rule-based, it provides an explainable and practical matching approach suitable for a junior design project.

REFERENCES

[1] Pallets Projects, "Flask Documentation," Available: https://flask.palletsprojects.com/

[2] SQLAlchemy, "SQLAlchemy ORM Documentation," Available: https://www.sqlalchemy.org/

[3] Tailwind Labs, "Tailwind CSS Documentation," Available: https://tailwindcss.com/

[4] Mozilla Developer Network, "JavaScript Guide," Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[5] TaskRabbit, "Service Marketplace Platform," Available: https://www.taskrabbit.com/

[6] Sheba.xyz, "On-demand Service Platform in Bangladesh," Available: https://www.sheba.xyz/

[7] Git, "Git Documentation," Available: https://git-scm.com/docs

[8] IEEE, "IEEE Editorial Style Manual," IEEE, 2023.